In [2]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
```

In [5]:
```
df = pd.read_csv("smartphone_overheating_pbl_final.csv")
df.head(20)
```

Out[5]:

	Battery_Temp_C	CPU_Temp_C	Ambient_Temp_C	CPU_Usage_Pct	GPU_Usage_
0	38.483571	50.244757	29.510183	35.980335	0.235
1	35.308678	32.757162	38.118758	63.838575	55.485
2	39.238443	46.603498	44.661971	86.824548	44.509
3	43.615149	37.046092	43.407392	33.084809	93.55
4	34.829233	39.380778	25.699396	14.717118	4.915
5	34.829315	42.380834	25.882436	48.244084	53.170
6	43.896064	41.044072	18.040804	7.731435	3.590
7	39.837174	28.591155	40.622195	48.847486	62.455
8	33.652628	53.054496	22.376601	71.303363	86.608
9	38.712800	51.075274	28.347276	56.147179	24.92
10	33.682912	40.829468	34.357792	93.105282	36.253
11	33.671351	49.475189	42.911324	59.013595	69.185
12	37.209811	38.818439	16.423387	21.710057	56.353
13	26.433599	34.142420	31.085475	34.836111	49.27
14	27.375411	42.435304	38.135659	94.702485	41.500
15	33.188562	32.648777	43.760902	83.610906	11.928
16	30.935844	46.872405	23.218168	25.871675	56.382
17	37.571237	36.379498	23.129438	89.259788	73.835
18	31.459880	29.199753	41.123397	70.049415	64.73
19	28.938481	12.982338	39.655585	91.173574	64.534

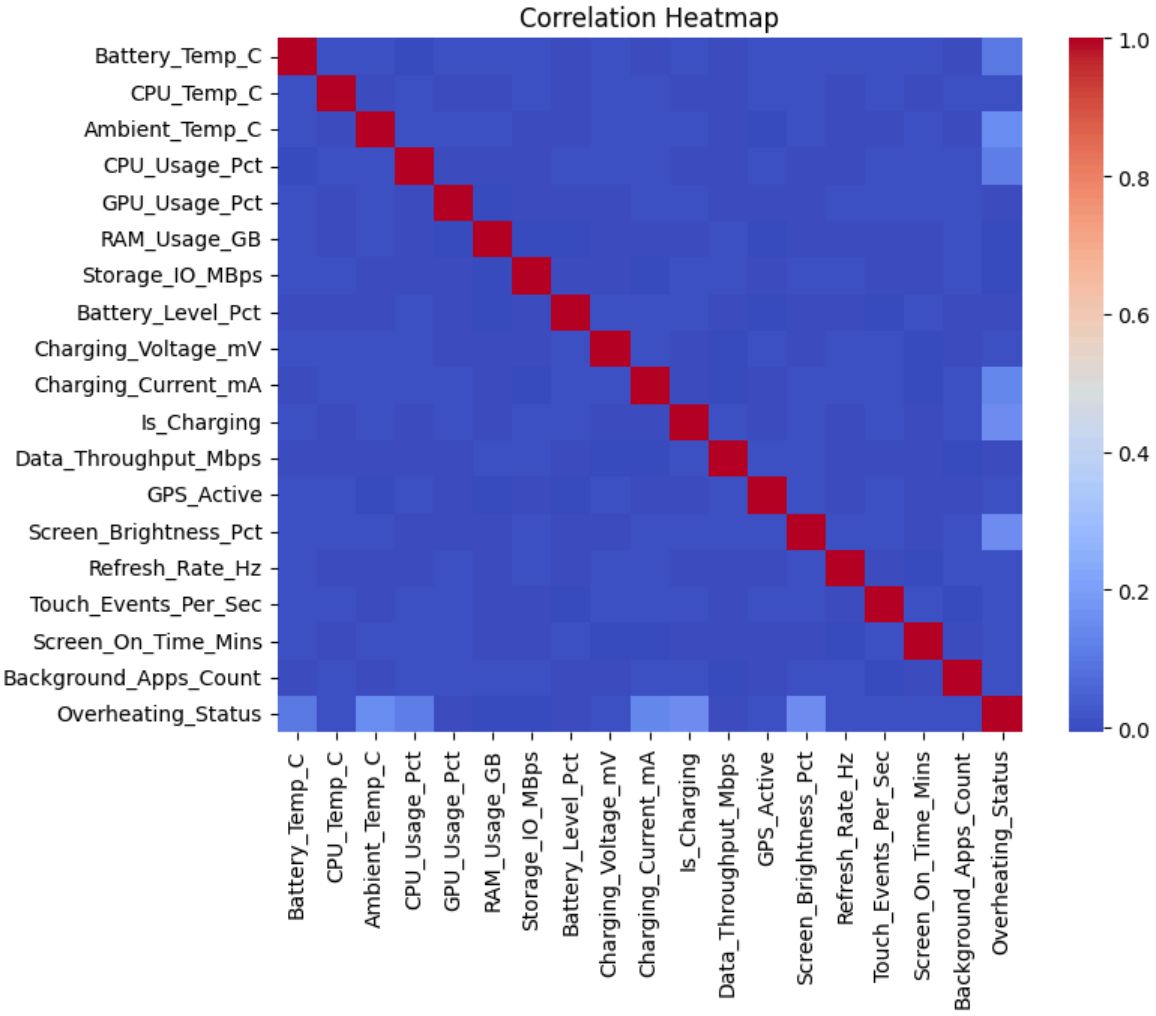
In [6]:
```
print(df.shape)
print(df.info())
print(df.describe())
```

```
(100000, 19)
<class 'pandas.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 19 columns):
 #   Column                 Non-Null Count   Dtype
---  ------                 --------------   -----
 0   Battery_Temp_C         100000 non-null  float64
 1   CPU_Temp_C             100000 non-null  float64
 2   Ambient_Temp_C         100000 non-null  float64
 3   CPU_Usage_Pct          100000 non-null  float64
 4   GPU_Usage_Pct          100000 non-null  float64
 5   RAM_Usage_GB           100000 non-null  float64
 6   Storage_IO_MBps        100000 non-null  float64
 7   Battery_Level_Pct      100000 non-null  float64
 8   Charging_Voltage_mV    100000 non-null  float64
 9   Charging_Current_mA    100000 non-null  float64
 10  Is_Charging            100000 non-null  int64
 11  Data_Throughput_Mbps   100000 non-null  float64
 12  GPS_Active             100000 non-null  int64
 13  Screen_Brightness_Pct  100000 non-null  int64
 14  Refresh_Rate_Hz        100000 non-null  int64
 15  Touch_Events_Per_Sec   100000 non-null  float64
 16  Screen_On_Time_Mins    100000 non-null  float64
 17  Background_Apps_Count  100000 non-null  int64
 18  Overheating_Status     100000 non-null  int64
dtypes: float64(13), int64(6)
memory usage: 14.5 MB
None
       Battery_Temp_C     CPU_Temp_C  Ambient_Temp_C  CPU_Usage_Pct  \
count   100000.000000  100000.000000   100000.000000  100000.000000
mean        36.004834      42.007848       30.000405      52.481313
std          5.004530       7.991515        8.647657      27.415485
min         13.671981       7.007658       15.000067       5.002959
25%         32.627531      36.610521       22.504927      28.819955
50%         36.013248      42.004009       29.994509      52.537091
75%         39.384573      47.399258       37.488799      76.201793
max         58.395421      78.496918       44.999838      99.998759

       GPU_Usage_Pct   RAM_Usage_GB  Storage_IO_MBps  Battery_Level_Pct  \
count  100000.000000  100000.000000    100000.000000      100000.000000
mean       50.118920       6.488855        19.996896          50.664096
std        28.816058       3.175092        19.930526          28.609198
min         0.000052       1.000058         0.000898           1.000503
25%        25.161342       3.746839         5.795317          25.841976
50%        50.164823       6.472891        13.866355          50.714720
75%        75.045469       9.234852        27.739419          75.474068
max        99.999831      11.999884       243.842741          99.999289

       Charging_Voltage_mV  Charging_Current_mA    Is_Charging  \
count        100000.000000        100000.000000  100000.000000
mean           3999.734113          2751.588422       0.298410
std             288.969684          1300.430144       0.457563
min            3500.000833           500.033687       0.000000
25%            3748.628393          1624.139779       0.000000
50%            4000.491216          2752.737139       0.000000
75%            4249.620511          3880.530405       1.000000
max            4499.977878          4999.960643       1.000000

       Data_Throughput_Mbps     GPS_Active  Screen_Brightness_Pct  \
count         100000.000000  100000.000000          100000.000000
```

```
mean              14.971077       0.198950              50.044220
std               15.044964       0.399212              29.178258
min                0.000156       0.000000               0.000000
25%                4.294661       0.000000              25.000000
50%               10.301856       0.000000              50.000000
75%               20.739295       0.000000              75.000000
max              167.953387       1.000000             100.000000

       Refresh_Rate_Hz  Touch_Events_Per_Sec  Screen_On_Time_Mins  \
count    100000.000000         100000.000000        100000.000000
mean         89.942400             30.009434           299.268339
std          24.465418             17.332825           172.725259
min          60.000000              0.000023             0.012055
25%          60.000000             15.062277           150.950111
50%          90.000000             29.953313           297.823630
75%         120.000000             45.036239           449.130974
max         120.000000             59.999963           599.994794

       Background_Apps_Count  Overheating_Status
count          100000.000000       100000.000000
mean               24.504720            0.055940
std                14.410664            0.229807
min                 0.000000            0.000000
25%                12.000000            0.000000
50%                25.000000            0.000000
75%                37.000000            0.000000
max                49.000000            1.000000
```

In [7]:

```
plt.figure(figsize=(8,6))
corr = df.corr()
sns.heatmap(corr, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```



In [8]:
```
X = df.iloc[:, :-1]
y = df.iloc[:, -1]
```

In [9]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

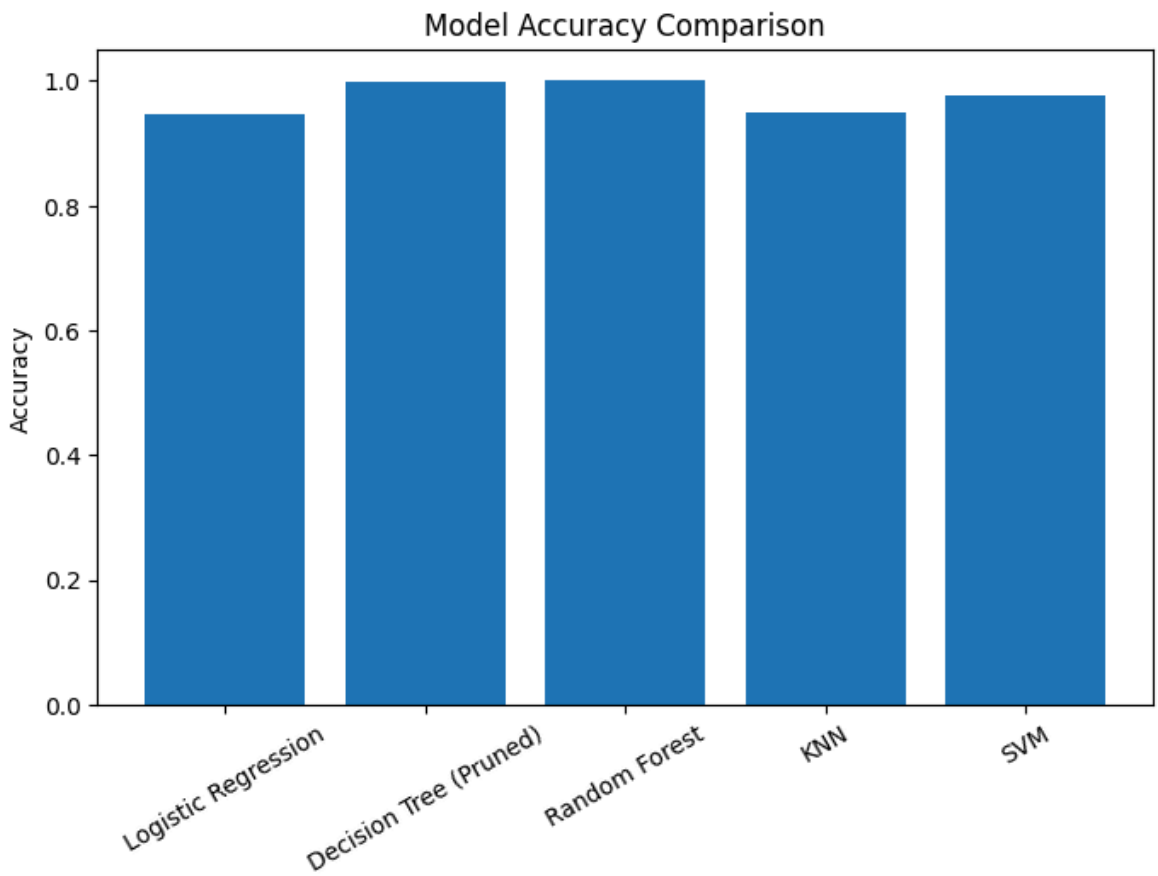
In [10]:
```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [11]:
```
models = {
    "Logistic Regression": LogisticRegression(max_iter=500),

    "Decision Tree (Pruned)": DecisionTreeClassifier(
        max_depth=5,
        min_samples_split=10
    ),

    "Random Forest": RandomForestClassifier(n_estimators=50),

    "KNN": KNeighborsClassifier(n_neighbors=5),

    "SVM": SVC()
}
```

In [14]:
```
results = {}

for name, model in models.items():
    print(f"\nRunning: {name}")

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    results[name] = acc

    print("Accuracy:", acc)
    print("Precision:", prec)
    print("Recall:", rec)
    print("F1 Score:", f1)
```

```
Running: Logistic Regression
Accuracy: 0.9474
Precision: 0.6697247706422018
Recall: 0.12966252220248667
F1 Score: 0.21726190476190477

Running: Decision Tree (Pruned)
Accuracy: 0.9974
Precision: 0.9990706319702602
Recall: 0.9547069271758437
F1 Score: 0.9763851044504995

Running: Random Forest
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0

Running: KNN
Accuracy: 0.94895
Precision: 0.6981132075471698
Recall: 0.16429840142095914
F1 Score: 0.26599568655643424

Running: SVM
Accuracy: 0.9764
Precision: 0.8699095022624435
Recall: 0.6829484902309059
F1 Score: 0.7651741293532338
```

In [15]:
```
plt.figure(figsize=(8,5))
plt.bar(results.keys(), results.values())
plt.xticks(rotation=30)
plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")
plt.show()
```



In [16]:
```
depths = [3, 5, 7]
best_acc = 0
best_depth = 0

for d in depths:
    model = DecisionTreeClassifier(max_depth=d)
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)

    print("Depth:", d, "Accuracy:", acc)

    if acc > best_acc:
        best_acc = acc
        best_depth = d

print("Best Depth:", best_depth)
```

```
Depth: 3 Accuracy: 0.9771
Depth: 5 Accuracy: 0.9974
Depth: 7 Accuracy: 0.99995
Best Depth: 7
```

In [17]:
```
best_dt = DecisionTreeClassifier(max_depth=best_depth)
best_dt.fit(X_train, y_train)
```

Out[17]:

▾ DecisionTreeClassifier ⓘ ?

▸ Parameters

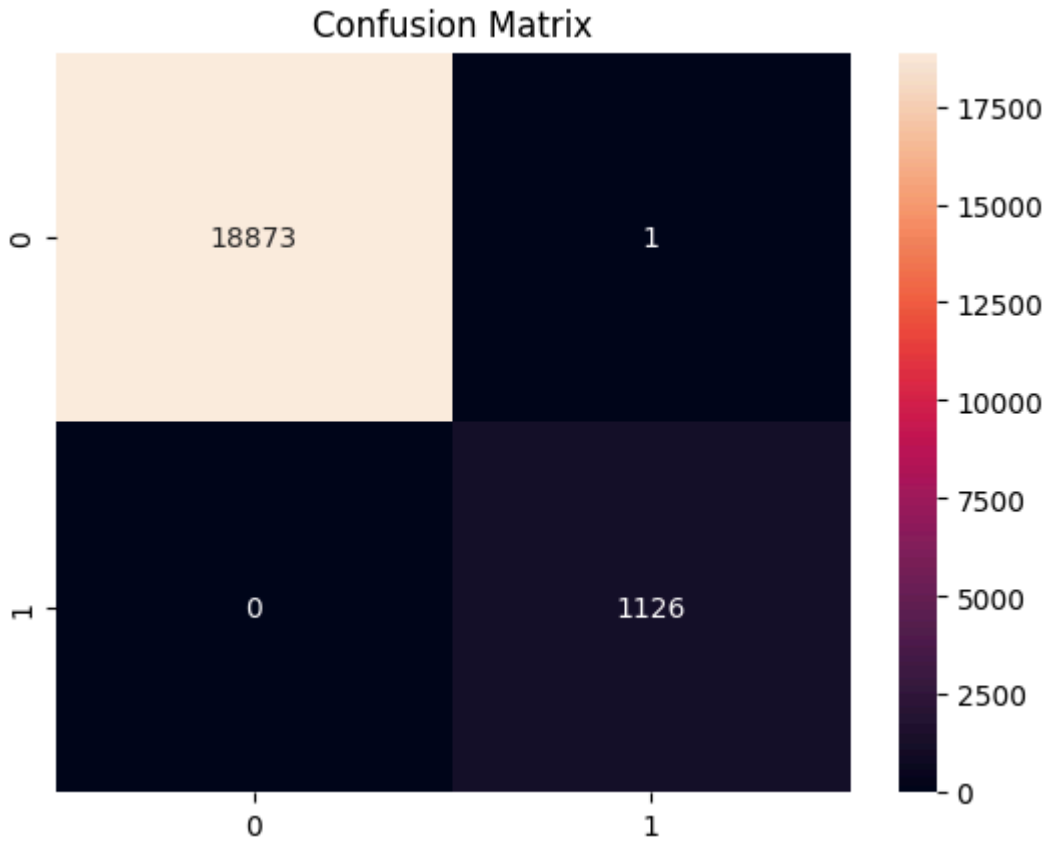
In [18]:
```
y_pred = best_dt.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))

print("\nClassification Report:\n", classification_report(y_test, y_pred)
```

```
Accuracy: 0.99995
Precision: 0.9991126885536823
Recall: 1.0
F1 Score: 0.9995561473590768

Classification Report:
               precision    recall  f1-score   support

           0       1.00      1.00      1.00     18874
           1       1.00      1.00      1.00      1126

    accuracy                           1.00     20000
   macro avg       1.00      1.00      1.00     20000
weighted avg       1.00      1.00      1.00     20000
```

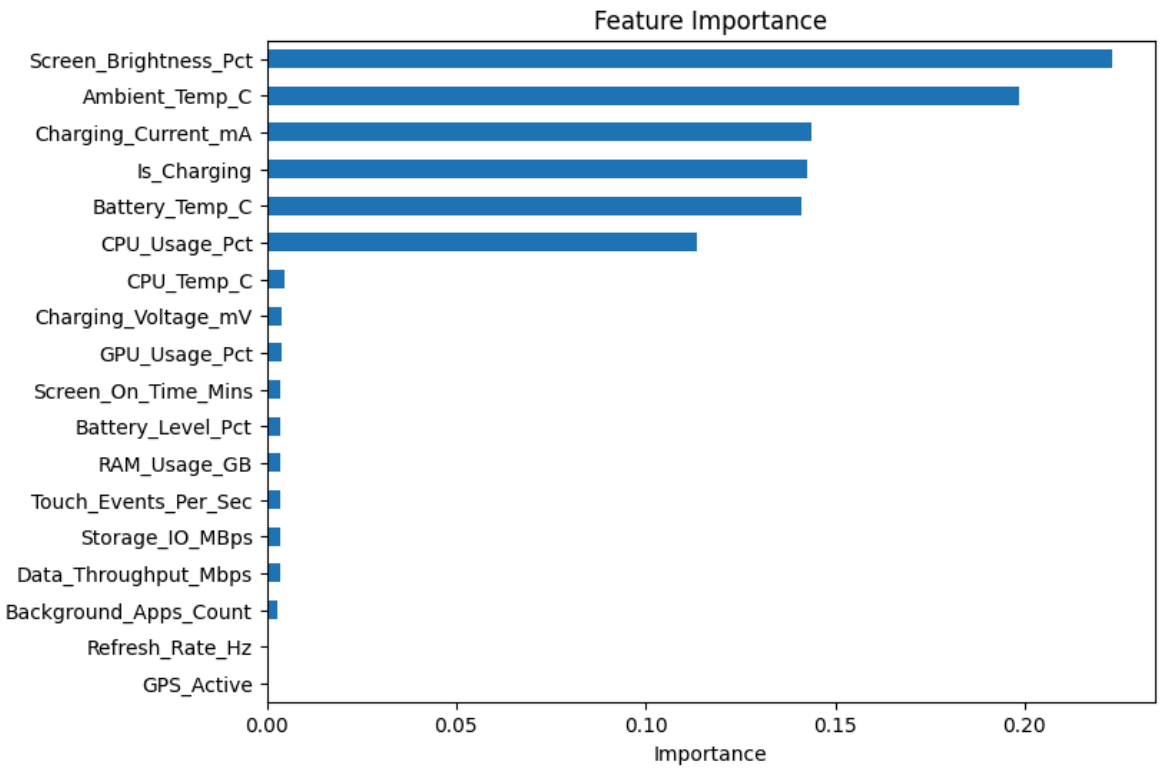
In [19]:
```
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[18873     1]
 [    0  1126]]
```

In [20]:
```
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.show()
```



In [21]:
```
rf = RandomForestClassifier(n_estimators=50)
rf.fit(X_train, y_train)

importances = rf.feature_importances_

feat_imp = pd.Series(importances, index=df.columns[:-1])
feat_imp = feat_imp.sort_values()

plt.figure(figsize=(8,6))
feat_imp.plot(kind='barh')

plt.title("Feature Importance")
plt.xlabel("Importance")
plt.show()
```


Feature Importance
Screen_Brightness_Pct
Ambient_Temp_C
Charging_Current_mA
Is_Charging
Battery_Temp_C
CPU_Usage_Pct
CPU_Temp_C
Charging_Voltage_mV
GPU_Usage_Pct
Screen_On_Time_Mins
Battery_Level_Pct
RAM_Usage_GB
Touch_Events_Per_Sec
Storage_IO_MBps
Data_Throughput_Mbps
Background_Apps_Count
Refresh_Rate_Hz
GPS_Active
0.00
0.05
0.10
0.15
0.20
Importance